jupyter SEPM_U9_eport_activity_ImprovingCodeQuality Last Checkpoint: 2 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[ ]:
```
# RobotController Code Quality Improvement

This notebook demonstrates improving code quality.
We will show a “bad” version of the code and an “improved” version, applying strategies like:
- Meaningful variable/function names
- Docstrings and type hints
- Logging and task history tracking

Below, you will see the before and after versions.
```

[ ]:
```
# BAD Version: messy and minimal comments
import random

class Robot:
    def __init__(self, id):
        self.id = id
        self.state = "idle"
        self.log = []

    def start(self):
        self.state = "idle"
        print("Ready")
x
    def deliv(self, item, f, t):
        if item in ["chair", "desk", "fridge"]:
            self.log.append(f"Rejected {item}")
            return "Can't deliver"
        success = random.choice([True, False])
        if success:
            self.log.append(f"Sent {item} to {t}")
            return f"Delivered {item} to {t}"
        else:
            self.log.append(f"Failed {item}")
            self.state = "error"
            return "Delivery failed"

r = Robot("R1")
print(r.deliv("chair", "Storage", "Classroom"))
print(r.deliv("book", "Library", "Classroom"))
```

[ ]:
```python
# IMPROVED Version: clean, readable, and maintainable
import random
from typing import List, Tuple

FORBIDDEN_ITEMS = ["chair", "desk", "fridge"]

class RobotController:
    """Robot controller for managing deliveries and logging tasks."""

    def __init__(self, id_: str):
        self.id = id_
        self.state = "IDLE"
        self.history: List[Tuple[str, str]] = []

    def start(self) -> None:
        """Initialize robot."""
        self.state = "IDLE"
        print("Robot ready.")

    def deliver_material(self, item: str, from_location: str, to_location: str) -> str:
        """Deliver materials if allowed."""
        if item.lower() in FORBIDDEN_ITEMS:
            self.history.append(("rejected", item))
            return f"Cannot deliver '{item}' - forbidden."
        success = random.choices([True, False], weights=[0.85, 0.15])[0]
        if success:
            self.history.append(("delivered", item))
            return f"Delivered {item} to {to_location}"
        else:
            self.state = "ERROR"
            self.history.append(("failed", item))
            self.recover()
            return f"Delivery of {item} to {to_location} failed."

    def recover(self) -> None:
        """Recover from error."""
        self.state = "RECOVERING"
        self.state = "IDLE"

# Demo
bot = RobotController("R1")
print(bot.deliver_material("chair", "Storage", "Classroom"))
print(bot.deliver_material("book", "Library", "Classroom"))
```

[ ]:
```markdown
## Comparison

- **BAD version**
  - Vague function names (`deliv`)
  - Minimal comments
  - No type hints
  - Hard-coded forbidden items

- **IMPROVED version**
  - Meaningful class and function names (`RobotController.deliver_material`)
  - Docstrings and type hints
  - Logging delivery history
  - Better error handling and recovery
  - Easy to read and maintain
```